

Assignment 2-Case Study

Stage 4 Lab Activities

Implementing the SmartCare Domain Layer

DESIGN FIRST -> AI PAIR PROGRAMMING -> REVIEW -> VERIFY | 1hour

A - Revisit Approved UML

Confirm responsibilities, attributes and relationships before coding.

B - Implement Patient: AI OFF

Implement Patient with type hints and basic validation.

C - Implement Practitioner: AI OFF

Implement Practitioner with identifier, name and specialty; no database logic.

D - Implement Appointment: AI ON

Give AI the approved Appointment UML, business rules and explicit constraints. Ask it to implement only Appointment and agreed enum/exception.

E - Review Generated Code

Check model consistency, unsupported features, public state mutation, unnecessary inheritance, invented dependencies and error handling.

F - Manual Behaviour Checks

Create valid objects, test invalid input, cancel a scheduled appointment and attempt an illegal repeated transition.

G - Refactor

Remove unnecessary code and make implementation simpler and design-consistent.

H - AI Engineering Log

Record prompt, generated contribution, decisions and verification evidence.

AI Prompt: The approved Appointment UML, business rules and explicit constraints were provided to the AI along with instructions on how to implement only the Appointment class and Appointment status enum.

Generated Contribution: AI generated the initial Appointment class structure, status enum, validation and appointment operations.

Decisions: The generated code was reviewed against the approved UML. Supported functionality was retained, while any unnecessary or unsupported functionality was rejected. Appointment status was protected from unrestricted public modification and status transitions were controlled through class methods.

Verification Evidence: Manual testing was performed by creating valid Patient, Practitioner and Appointment objects, providing invalid input, cancelling a scheduled appointment and attempting to cancel the appointment a second time. The tests were also validated by running the code and ensuring it ran successfully. The tests confirmed valid object creation, error handling and cancellation.

Refactoring: The final implementation was simplified by removing unnecessary functionality and ensuring that the Appointment class remained consistent with the approved SmartCare domain model.

Suggested AI prompt

Act as a Python pair programmer. Implement only the Appointment class from the approved SmartCare UML. Use type hints and an AppointmentStatus enum. Cancelled appointments remain as objects. Do not add database, UI, notification or service classes. Protect status transitions and explain any decision not directly visible in the UML.

Reflection

Which AI-generated part did you modify or reject? Why? How did the approved design constrain the AI?

The AI-generated Appointment class was reviewed against the approved SmartCare UML and business rules. The main parts retained were the Appointment status enum, appointment attributes, validation, cancellation method and protected status transitions because these were consistent with the approved design.

I identified that the generated code did not independently implement the double-booking rule. I did not add extra classes or dependencies to solve this because the approved UML did not define an appointment manager, database or other service for managing multiple appointments. Adding these would have expanded the design beyond the agreed domain model.

The approved design constrained the AI by limiting the implementation to the Appointment class and Appointment status enum. This prevented unsupported features such as databases, notification systems, controllers and additional management classes from being introduced. Objects were then manually tested along with invalid input, appointment cancellation and an illegal repeated cancellation to verify the behaviour of the implementation.